

Auto In-room Provision System Group 8 Project Report

Bryan Li – SN 25003743
Yan Pei Zhu – SN 25103352
Nolan Yu – SN 25113715

March 10, 2026

Contents

1	Introduction	2
1.1	Project Scope and Limitations	3
2	Theoretical Background	4
2.1	Schematic Diagram	4
2.2	System Overview	5
2.3	Flowcharts	6
2.4	PID Tuning Strategy	9
2.5	Mechanics of the System	11
2.6	Sensor Principles	13
3	Components	19
3.1	Components Overview	19
3.2	Power Budget Analysis	19
4	Project Summary	22
5	Electronics and Electrical Design	22
5.1	Electrical Schematics	23
5.2	Detailed Specifications of Key Components	24
5.3	Mechanical Structure and Actuation Design	27
6	Experimental Evaluation	27
7	Discussion and Conclusion	27
8		27

1 Introduction

The system developed in this project is a two-wheeled autonomous mobile delivery robot designed for short-distance indoor delivery tasks; equipped with a top-mounted cargo basket providing a secure platform for carrying lightweight payloads such as food, parcels, or supplies.



Figure 1: Commercial Delivery Robot¹

We took inspiration from commercial hotel delivery robots, both designed to transport delivery items with collision detection and autonomous movement in indoor buildings. The key differences include navigation methods and environment integration, our system is unable to support lift integration for multi-floor delivery tasks and can only follow a prepared route due to the processing and sensing capabilities associated with our given components.

Autonomous delivery robots have gained increasing prominence in commercial and industrial environments due to their ability to improve operational efficiency and reduce human labour requirements. Commercial hotel delivery robots incorporate advanced technologies such as SLAM-based navigation, multi-sensor fusion, elevator integration, and cloud-based fleet management systems². These systems are designed to operate autonomously in dynamic, multi-floor hospitality environments while interacting safely with guests and infrastructure. However, incorporating such abilities significantly increases system complexity and deployment cost.

In contrast, the proposed Arduino R4-based delivery robot adopts a structured navigation strategy using an infrared (IR) sensor for predefined line tracking and a LiDAR for obstacle detection. Unlike SLAM³-based systems that rely on probabilistic localisation and mapping algorithms, the proposed design employs predefined trajectory tracking, reducing computational requirements and minimising computational complexity and efficiency. The position error relative to the line is computed from the sensor array readings and regulated using a PID control strategy⁴, this approach enhances reliability in controlled indoor environments where routes are fixed and repetitive, decreasing reliance on manual labour.

1.1 Project Scope and Limitations

To clearly distinguish the intended functionality of the proposed system from advanced commercial implementations, the scope of this project is explicitly defined.

In Scope

The system is designed to:

- Perform predefined indoor navigation via infrared line tracking.
- Detect obstacles using LiDAR-based distance thresholding.
- Maintain orientation stability (pitch) using IMU-based feedback.
- Utilise a multi-layer PID control framework to regulate wheel speed, directional orientation, pitch, and linear motion.
- Operate autonomously within structured, controlled indoor environments.

Out of Scope

The following capabilities are intentionally excluded due to hardware, computational, and project constraints:

- SLAM-based localisation and real-time environment mapping.
- Multi-floor navigation or elevator integration.
- Dynamic path planning in unstructured environments.

The exclusion of these advanced functionalities ensures that the system remains computationally feasible for implementation on the Arduino R4 platform while maintaining clarity of design objectives and technical focus. Our system integrates five PID controllers to provide reliable robot motion across predefined reflective lines using more affordable and maintainable hardware.

2 Theoretical Background

2.1 Schematic Diagram

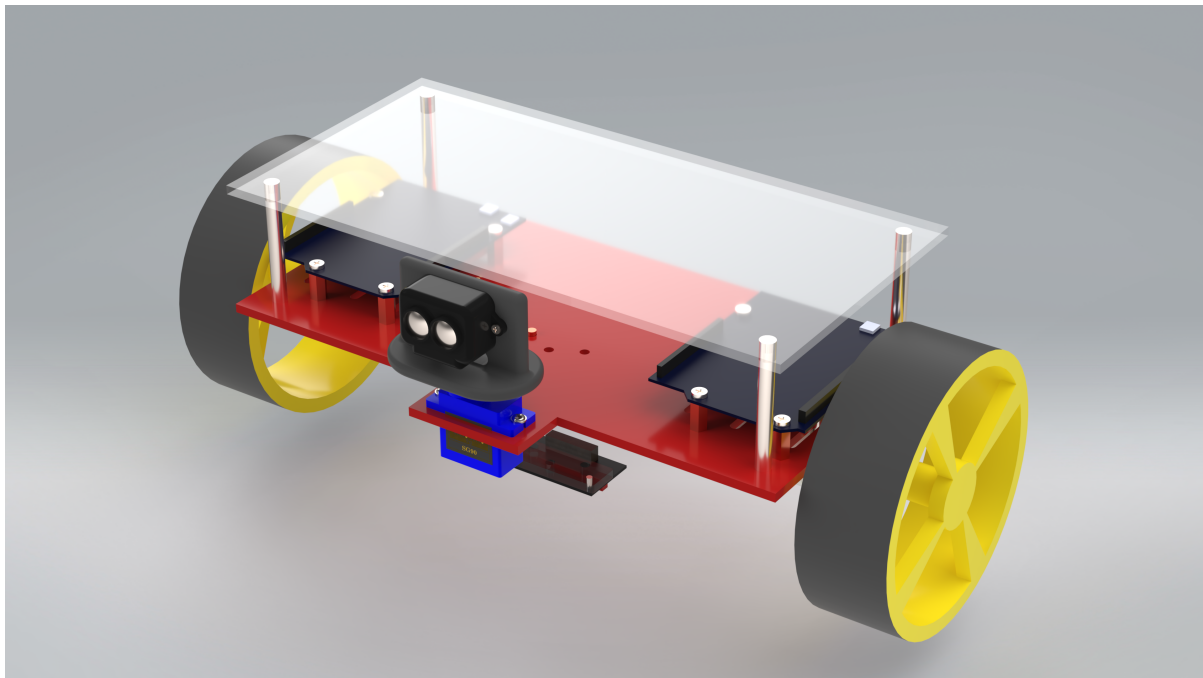


Figure 2: Schematic 3D Model

Figure 2 illustrates the mechanical structure of the robot. The chassis is fabricated from 5 mm laser-cut acrylic and consists of a two-layer configuration. The lower layer supports the drive motors, wheel assemblies, motor driver circuitry, IMU, and battery which provide a stable structural base. These will be connected to the master board microcontroller.

To improve static and dynamic stability through balanced mass distribution, the slave board microcontroller is positioned on the opposite side of the chassis relative to the master controller.

The upper layer is mechanically separated to house control electronics including the LiDAR, servo, OLED displays, and IR sensor. The upper structural layer creates a functional platform for payload placement; to counteract variations in the centre of mass resulting from payload changes and distributions, the control system adjusts the target pitch, thereby preserving dynamic stability.

The distributed PCB and dual-deck chassis configuration promotes a modular system architecture, facilitating subsystem isolation, simplified maintenance, and scalable hardware expansion. By positioning the heavier components on the bottom deck, the centre of mass is lowered; this optimises static and dynamic stability by reducing overturning moments during acceleration and differential turning. As a result, the robot exhibits improved resistance to tipping and enhanced motion control performance.

2.2 System Overview

The system establishes a seamless connection between the controller and the robot, enabling intuitive movement control. Communication is handled via wireless WiFi communication, utilising the built-in capabilities of the Arduino Uno R4.

The slave Arduino, Figure 5, acquires and processes local sensor data from the IR sensor and user-inputted movement direction. The processed outputs are communicated to the master Arduino through the I²C bus. Within this configuration, the master Arduino controls the communication sequence and polls the slave for updated data. The received information is subsequently utilised as input for the PID controllers to control the angular velocity of the motors for a balanced and steady movement.

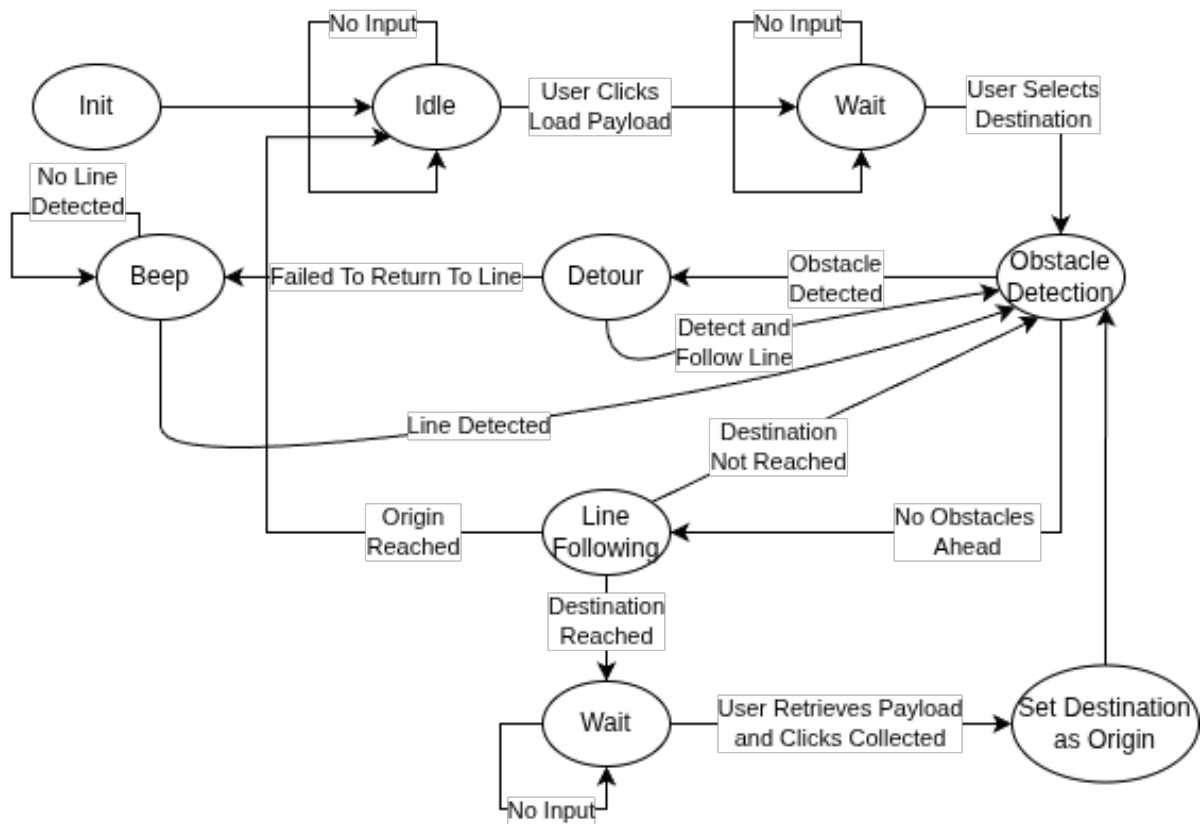


Figure 3: State Transition Diagram

The diagram 3 illustrates the states and transitions of an autonomous line-following robot, defining how it responds to user interactions, obstacle detection, and line tracking.

2.3 Flowcharts

Master board

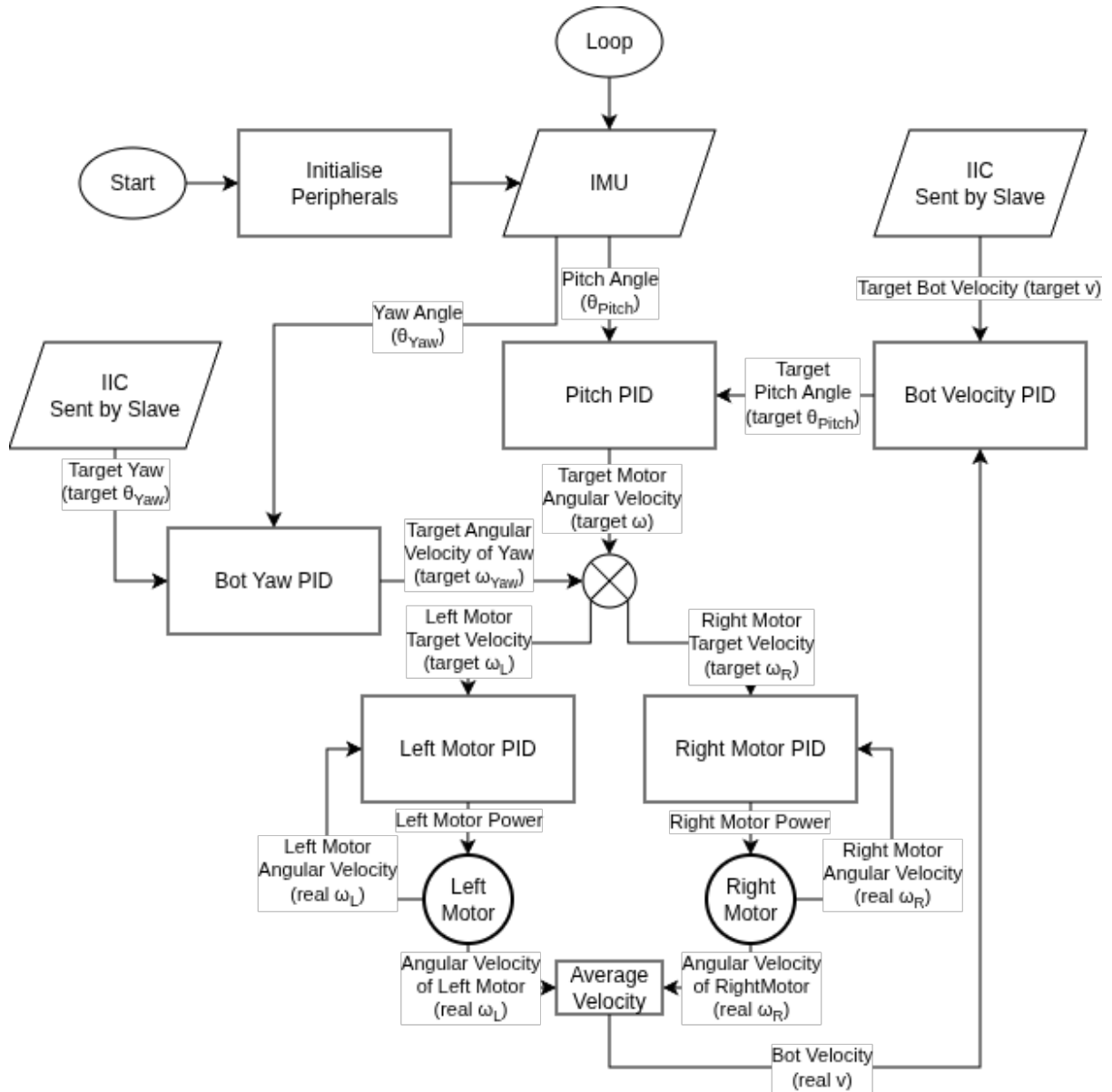


Figure 4: Master Board Flowchart

Description

To improve the stability of the wheels and the balance of the robot, five PID controllers are used to achieve the following:

- Dynamic balance (pitch stabilisation)
- Directional control (yaw regulation)

- Linear velocity control
- Individual motor speed regulation

Each motor is equipped with an encoder that provides real-time angular velocity feedback. The left and right motors' angular velocities (ω_L and ω_R respectively) are continuously measured and transmitted to the control system. These two values are averaged to estimate the overall linear velocity of the robot, since the platform is a differential drive system, the mean of the angular velocities provides an accurate representation of the robot's forward translational speed, which is then used as feedback in the velocity control loop.

Slave Board

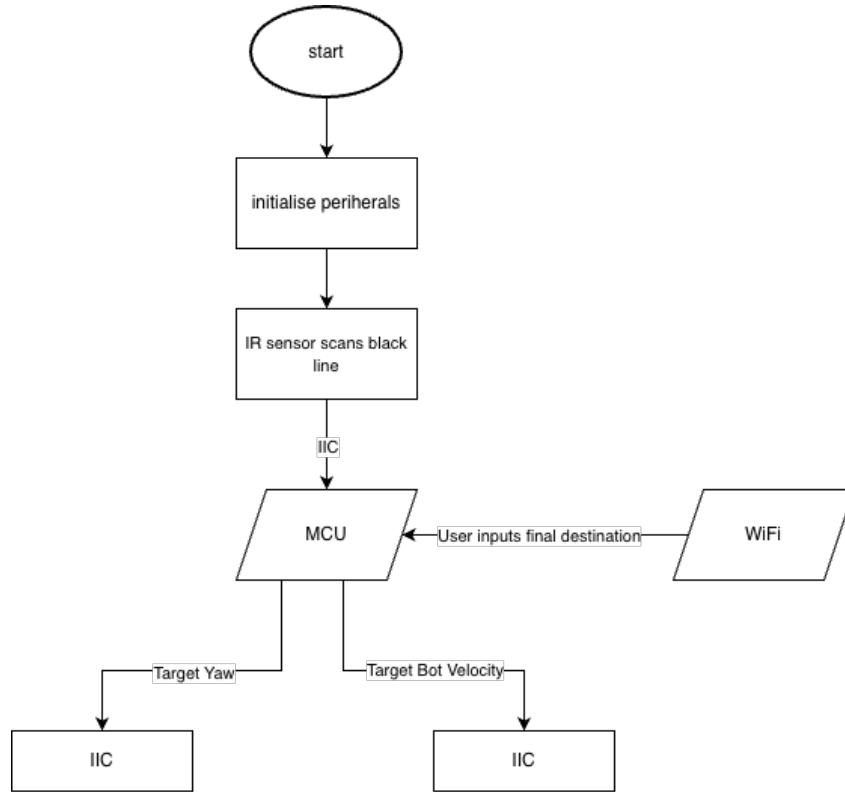


Figure 5: Slave Arduino

The slave board acquires line-tracking data via the IR sensor, processes navigation logic locally, and communicates motion references to the master controller. This distributed architecture reduces computational burden on the motor-control MCU and enforces modular separation between sensing and low-level actuation.

Status Display

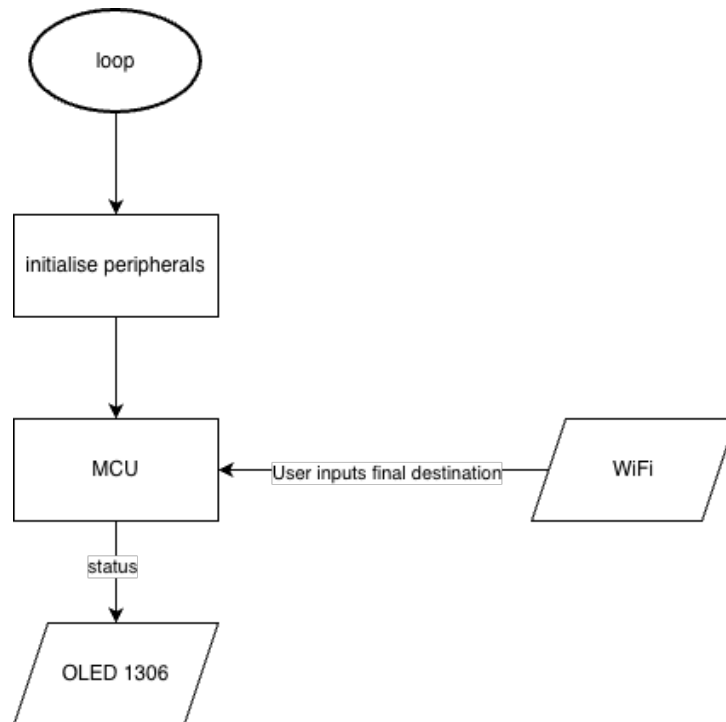


Figure 6: Slave Status Display

The system interface (OLED 1306) displays the current operational status of the robot, such as navigating to the destination, having reached the final destination, or returning to the starting point. Line tracking is achieved using an infrared (IR) sensor that detects the presence of black tape on the ground surface.

The robot follows this predefined path as the control algorithm continuously processes the sensor output to maintain alignment with this route. The destination is determined when the sensor detects a significantly larger continuous area of black tape relative to the normal line width, indicating a predefined destination. When this endpoint has been reached, the robot remains stationary until the user clicks , then returning to its starting point.

Collision Detection

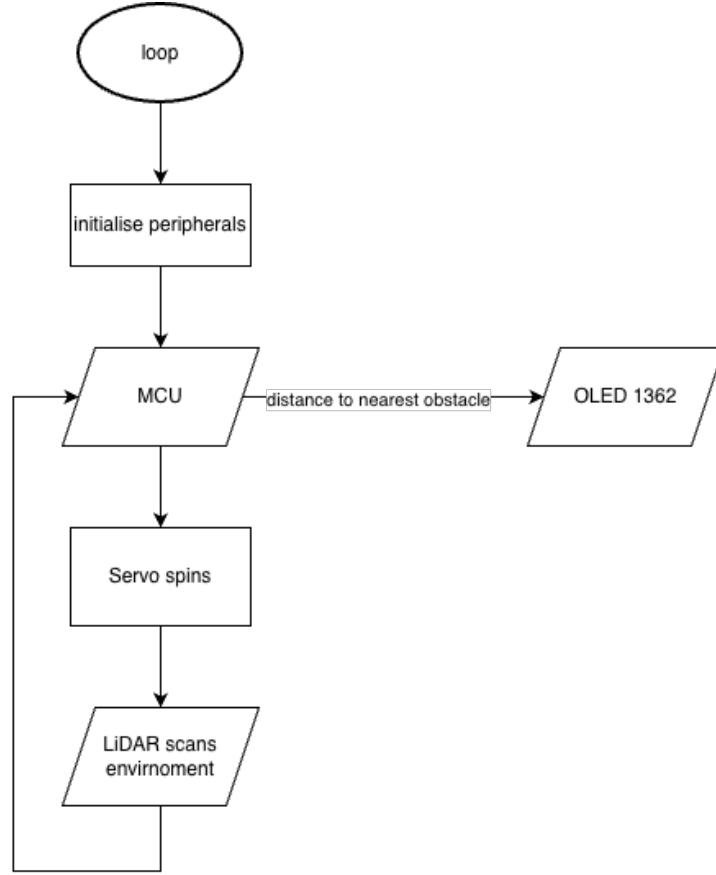


Figure 7: Slave Collision Detection

To extend the sensing capability of the system beyond single-direction distance measurement, a TF-LUNA LiDAR sensor is mechanically mounted onto a Mini SG90 180-degree servo motor. As the SG90 servo actuates, the LiDAR sensor rotates synchronously, allowing distance measurements to be obtained at discrete angular intervals, effectively turning the LiDAR into a radar.

The OLED 1362 displays the radar image that is updated as the LiDAR moves in 1° intervals, this design improves environmental perception while maintaining low system complexity and minimal hardware cost.

2.4 PID Tuning Strategy

In our system, all higher-level controllers ultimately influence the same pair of drive motors, the subsystems are dynamically coupled. As a result, modifications to one controller alter the effective dynamics experienced by others.

The Ziegler-Nichols method was deemed unsuitable for this system as it assumes a stable and weakly coupled system for parameter identification. The prototype and its constituent

subsystems exhibit instability under open-loop conditions, sensitivity to noise, and strongly coupled; hence, this is not a good approach to find reliable PID values.

Therefore, a structured sequential tuning procedure was adopted rather than simultaneous multi-loop adjustment.

Motor Speed Control

The tuning process commenced with the inner motor speed controllers. The left and right wheel loops were tuned independently using encoder feedback to achieve a fast, well-damped transient response with insignificant steady-state error. Achieving matched transient and steady-state characteristics across both motors was essential to avoid directional bias and maintain symmetry in differential-drive motion. By stabilising the motor loops at high bandwidth, a predictable and well-conditioned inner plant is created for outer-loop regulation.

Speed Regulation

After the motor loops were stabilised, the pitch controller was tuned using inertial measurement feedback. Since pitch correction modifies motor torque commands directly, this loop was designed to operate at a lower bandwidth than the motor controllers, preventing the excitation of fast inner-loop dynamics.

Yaw Control

The yaw controller was subsequently tuned to regulate planar orientation; since yaw control inherently interacts with translational motion, gain was adjusted in small intervals to prevent oscillatory coupling with the inner motor loops.

Translational Velocity Control

This loop regulates the average wheel velocity and therefore operates at the lowest bandwidth within the hierarchy. By ensuring that velocity control dynamics are significantly slower than yaw and pitch regulation, interference between loops is minimised.

Bandwidth Separation and Robustness Considerations

To preserve closed-loop stability in the cascaded architecture, bandwidth separation was enforced between successive control layers. Let ω_m denote the closed-loop bandwidth of the motor speed controllers. The hierarchy was designed such that

$$\omega_{\text{velocity}} < \omega_{\text{yaw}} < \omega_{\text{pitch}} < \omega_m. \quad (1)$$

This hierarchical separation limits dynamic interaction, reduces cross-coupling effects, and enhances robustness against modelling inaccuracies, preventing oscillatory behaviour, and sensor noise.

2.5 Mechanics of the System

Differential Drive Motion

The robot employs a differential drive mechanism consisting of two independently driven wheels mounted on either side of the chassis. The robot's motion is controlled by varying the angular velocities of the left and right wheels.

The forward linear velocity v of the robot is determined by

$$v = \frac{r}{2}(\omega_R + \omega_L)$$

where

- r — wheel radius
- ω_R, ω_L — angular velocities of the right and left wheels

Straight-line motion is achieved when both wheels rotate at equal angular velocities, conversely, a difference in wheel speeds introduces rotational kinematics about the robot's centre.

The angular velocity (yaw rate) of the robot about its vertical axis is

$$\omega_{\text{Yaw}} = \frac{r}{L}(\omega_R - \omega_L)$$

where

- L – Distance between the two wheels

Planar Motion of the Robot

The robot's position in the global coordinate frame is described by

$$\begin{aligned}\dot{x} &= v \cos \theta_{\text{Yaw}} \\ \dot{y} &= v \sin \theta_{\text{Yaw}}\end{aligned}$$

where

- (x, y) – Position of the robot
- θ_{Yaw} – Orientation angle (heading)

The equations describe the robot's motion based on wheel speeds.

Self-Balancing (Inverted Pendulum Behavior)

The system can be modelled as an inverted pendulum mounted on a mobile base, where stability is achieved by continuously adjusting wheel motion to regulate the body pitch angle.

The simplified nonlinear dynamics of the balancing motion can be expressed as:

$$\tau = I\ddot{\theta}_{\text{Pitch}} + b\dot{\theta}_{\text{Pitch}} + mg\ell \sin \theta_{\text{Pitch}}$$

where

- θ_{Pitch} — pitch angle of the robot
- I — moment of inertia
- b — damping coefficient
- m — mass of the robot
- ℓ — distance from the wheel axis to the center of mass
- τ — control torque generated by the motors

For small deviations from the upright position, the approximation $\sin \theta \approx \theta$ can be applied, yielding a linearised model

$$I\ddot{\theta}_{\text{Pitch}} + b\dot{\theta}_{\text{Pitch}} + mg\ell\theta_{\text{Pitch}} = \tau.$$

Assuming zero initial conditions, taking the Laplace transform gives

$$(Is^2 + bs + mg\ell)\Theta_{\text{Pitch}}(s) = \tau(s).$$

Hence, the transfer function from the motor torque input to the pitch angle output is

$$G(s) = \frac{\Theta_{\text{Pitch}}(s)}{\tau(s)} = \frac{1}{ls^2 + bs + mg\ell}. \quad (2)$$

This model describes the system's response to motor-generated torque, forming the basis for closed-loop balance control using IMU feedback.

Encoder-Based Velocity Measurement

Wheel angular velocity is measured using incremental encoders attached to the motor shafts. The angular velocity (ω) is calculated from the change in encoder counts over time,

$$\omega_{\text{L/R}} = \frac{2\pi}{N} \cdot \frac{\Delta C}{\Delta t}$$

Where

- N — counts per revolution of the encoder
- ΔC — change in encoder counts
- Δt — time interval

This measurement provides real-time feedback for the motor control system and enables precise motion regulation.

2.6 Sensor Principles

Motor Encoder Principle

The Pololu gearmotor is integrated with a quadrature encoder that uses a magnetic Hall-effect sensing mechanism to detect rotation. A magnetic disc is installed into a rear protrusion of the shaft and rotates with it. Two Hall-effect sensors are included, which detect magnetic-field variations and output a digital square wave.

Quadrature Encoder Output

An encoder outputs two quadrature square-wave signals, Channel A and Channel B, with a fixed phase offset of $\frac{\pi}{2}$. The output signals toggle between 0V and V_{cc} , with V_{cc} set to 5V, allowing direct interfacing with the GPIO inputs of the Arduino Uno R4 WiFi, which support both 3.3 V and 5 V logic levels.

Resolution

As specified in the motor datasheet, the encoder resolution is 48 counts per revolution (CPR), representing 48 detectable signal transitions per complete revolution of the motor shaft before gearbox reduction.

Direction Detection

The key concept of detecting direction is by recognising the phase difference of two channels.

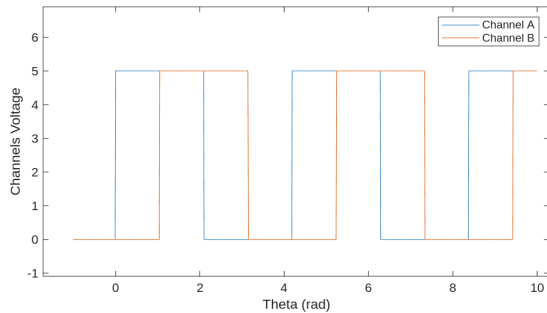


Figure 8: Clockwise Responses

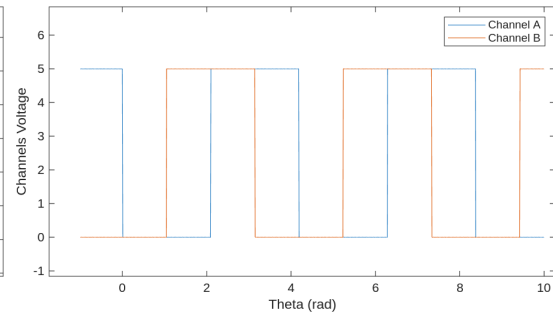


Figure 9: Anti-Clockwise Responses

As shown in Figure 8 and 9, the phase relationship depends on the direction of rotation, when it rotates clockwise, Channel A leads Channel B. The whole cycle can be represented as Channel A pulls up → Channel B pulls up → Channel A pulls down → Channel B is pulled down.

Conversely, when it rotates counterclockwise, Channel B leads Channel A. The whole cycle can be represented by Channel B pulls up → Channel A pulls up → Channel B pulls down → Channel A pulled down.

Algorithm Designs

Due to computational constraints of the Arduino R4, execution overhead is introduced by frequent interrupt handling in a real-time control environment. The system is limited to reliably handling two interrupt signals, connecting Channel A from both motors to the digital pins PD2 and PD3. The interrupt modes are set to CHANGE, meaning the ISR function is triggered on both rising and falling edges of the input signal.

```
1 void Motor::Motor() {
2     attachInterruptParam(
3         digitalPinToInterrupt(P_CHANNEL_A),
4         Motor::isr, CHANGE, this);
5 }
6
7 void Motor::isr(void* ins) {
8     if (digitalRead(P_CHANNEL_A) != digitalRead(P_CHANNEL_B)) {
9         (Motor* (ins))->count ++;
10    } else {
11        (Motor* (ins))->count --;
```

```

12     }
13 }

```

Since Channel A and Channel B maintain a fixed 90 phase offset, their instantaneous logic levels follow a predictable pattern that depends on the direction of rotation.

Table 1 presents all four possible transitions of Channel A and the corresponding states of both channels at the instant the ISR is triggered.

Table 1: Channel states at each Channel A transition under CHANGE interrupt mode.

Direction	Transition	A	B	A $\neq$ B	Action
Clockwise	Rising	HIGH	LOW	TRUE	count++
Clockwise	Falling	LOW	HIGH	TRUE	count++
Anti-Clockwise	Rising	HIGH	HIGH	FALSE	count--
Anti-Clockwise	Falling	LOW	LOW	FALSE	count--

As shown in Table 1, the condition $A \neq B$ is always true during clockwise rotation and false during anti-clockwise rotation, regardless of whether the transition is rising or falling. Therefore, a single comparison `digitalRead(A) != digitalRead(B)` is sufficient to determine the direction at every signal transition.

In our system, interrupts are triggered exclusively by transitions on Channel A. At the instant of each interrupt, the state of Channel B is read and evaluated to infer the direction of rotation based on the quadrature relationship. The raw CPR shown in the data sheet accounts for both channels, since Channel A serves as the sole interrupt source for transition detection, the effective CPR is reduced to 24.

Hence, the angular velocity after its gearbox is:

$$v = \frac{2\pi}{\text{CPR} \times \text{Gear Ratio}} * \frac{d}{dt} \text{count}, \quad \text{CPR} = 24, \text{Gear Ratio} = 9.68$$

Servo PWM Principle

The servo motor⁵ is controlled using a pulse-width modulated (PWM) control signal with a fixed period of 20 ms (50 Hz). Unlike conventional duty-cycle-based PWM used for power modulation, servo position is determined by pulse width within each period. A pulse width of approximately 1.5 ms corresponds to the neutral position (θ), while pulse widths of approximately 1.0 ms and 2.0 ms correspond to the lower and upper rotational limits of the servo (approximately 90° and +90°) respectively.

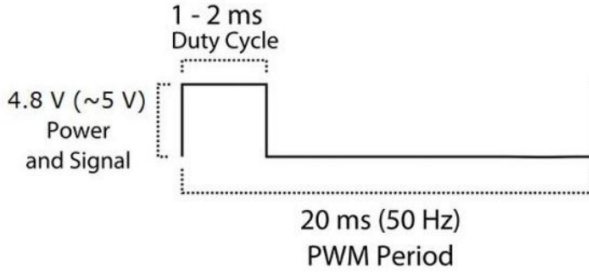


Figure 10: Servo PWM Specifications

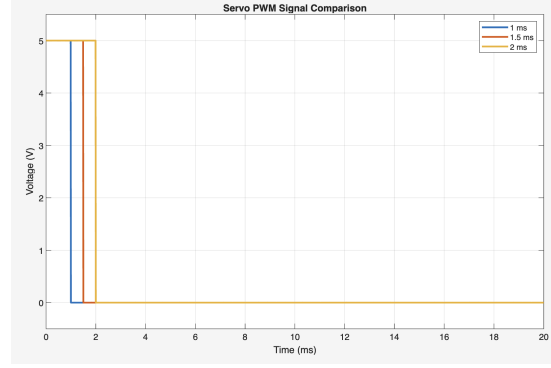


Figure 11: Servo PWM Signal Comparison

The servo internally interprets the pulse width as a position reference and employs closed-loop feedback to drive the output shaft to the commanded angle.

This control method is implemented in the collision detection subsystem to create and update the radar image, and controls the angular intervals that the LiDAR is rotating at. Bidirectional motion is implemented within the mechanical range to prevent cumulative cable twisting.

LiDAR Ranging Principle

The TF-Luna LiDAR measures distance using a phase-shift Time-of-Flight (ToF) method. A modulated near-infrared signal is emitted toward the target, and the reflected signal returns after propagating to the object and back to the sensor.

Both transmitted and reflected signals have the same modulation frequency; however the reflected signal experiences a phase delay due to the finite propagation time of light. This phase difference, denoted as $\Delta\phi$, is proportional to the round-trip travel distance. The distance is computed from the phase difference as

$$D = \frac{c}{2} \cdot \frac{\Delta\phi}{2\pi f},$$

where c is the speed of light and f is the modulation frequency.

Key Specifications and Practical Notes

According to the manufacturer's specifications, the TF-Luna has an operating range of approximately 0.2–8 m, with a nominal resolution of 1 cm. The reliability of measurements may be compromised in close-proximity scenarios, for surfaces with low reflectivity, or under significant ambient light interference. The LiDAR module has been integrated into the system for obstacle detection by using a predefined distance limit for decision-making and triggering an avoidance behaviour when the distance falls below a set limit.

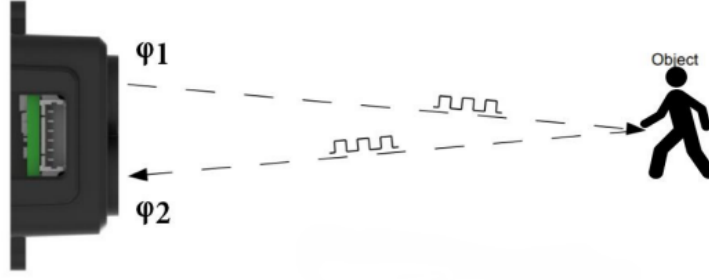


Figure 12: LiDAR ToF Method Illustration⁶.

See Figure 12, the emitted infrared signal reflects from the object and returns with a phase delay $\Delta\phi$ proportional to the distance.

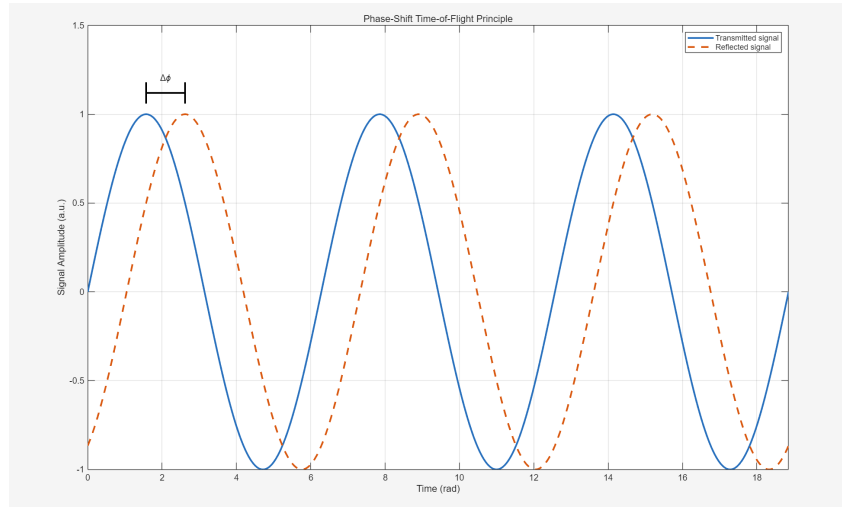


Figure 13: ToF Phase-shift Waveform

See Figure 13, the transmitted and reflected signals have the same frequency but differ in phase by $\Delta\phi$ due to the propagation delay.

I²C Communication

The communication between the master board and slave board is by I²C. As the program will be blocked when I²C is communicating and the real-time performance requirements of the system is high, the duration of each communication must be as short as possible.

When the I²C is initialised as **fast mode**, its communication's frequency is 400kHz⁷, transmitting one byte of data requires nine clock periods, eight bits and one ACK.

Hence, the transmission rate can be calculated by

$$\frac{400\text{kHz}}{8 + 1} \approx 44\text{KB/s}$$

Package Design

In this design, the slave board controls the target velocity and target yaw angle of the bot. Thus, the package contains at least two float variables **target vel** and **target yaw**. To ensure the the data remains accurate when received by the master board, a **check sum** is added. The last bit of **check sum** represents whether the data has been processed.

```
1 #pragma pack(push, 1)
2 struct slave_to_master_iic_data_t {
3     float target_vel;
4     float target_yaw;
5     uint8_t check_sum : 7;
6     uint8_t is_new_data : 1;
7 };
8 #pragma pack(pop)
```

This structure occupies a total of 9 bytes; including two float (8 bytes), a **check sum** byte, and a **is new data** flag (1 byte). The expected transmission time is:

$$t = \frac{9\text{bytes}}{44\text{KB/s}} \approx 205\text{us}$$

The `#pragma pack(push, 1)` directive instructs the compiler to disable default memory alignment, forcing all struct members to be packed at 1-byte boundaries. Without this directive, the compiler would pad the `uint8_t` members to align with 4-byte boundaries, increasing the total struct size from its expected value to 12 bytes.

3 Components

3.1 Components Overview

The system architecture consists of the following hardware components required for sensing, actuation, control, and user interaction:

Category	Components Name	Model	Usage
Microcontroller	Arduino R4	RA4M1	Master Board
Microcontroller	Arduino R4	RA4M1	Slave Board
Microcontroller	ESP32	ESP32-S3-MINI-1-N8	WiFi Board
Mechanical Input	Modulino Movement	LSM6DSOXTR	Motion Sensor (IMU)
Indicator	Modulino Buzzer	PKLCS1212E4001	Audio Indicator
Motor Controller	Motoron	M3S550	Motor Driver
Actuator	Pololu Geared Motor	4882 (560 RPM, 12V)	DC Motor ($\times 2$)
Actuator	Mini Servo	SG90 (180° Resin Gear)	Rotate LiDAR
Sensor	LiDAR	TF-LUNA	Obstacle Detection
Sensor	IR Sensor	QTRX-HD-09RC	Path Detection
Display	OLED Display	1306	Visual Output
Display	OLED Display	1362	Visual Output

Table 2: Components list with categories and models

3.2 Power Budget Analysis

A power budget analysis was performed to estimate the system’s current consumption under both worst-case and typical operating conditions and to verify that the selected power source can meet the required operating time. The robot includes two DC motors, two Arduino UNO R4 WiFi microcontrollers, a TF-Luna LiDAR, an SG90 servo, two OLED displays (SSD1306 and SSD1362), and several low-power sensors (see Table 2).

Current values are taken from manufacturer datasheets where available. For components with load-dependent consumption, such as the Arduino boards and OLED displays, conservative typical values are adopted. The OLED currents are derived from the SSD1306 and SSD1362 electrical specifications, which depend strongly on display content and brightness.

The system is assumed to be powered by a 3S LiPo battery (nominal 11.1 V, 2200 mAh), which directly supplies the DC motors, while low-voltage electronics are powered through DC–DC converters.

Peak Current Budget (Worst-Case)

For a conservative estimate, actuators are assumed to operate at stall or peak load, while controllers and sensors use their maximum specified currents. This scenario represents a worst-case transient condition rather than continuous operation.

Table 3: Peak current budget (worst-case)

Component	Qty	Peak per unit	Subtotal
Pololu 4882 DC motor (stall)	2	0.90 A	1.80 A
Arduino UNO R4 WiFi (peak)	2	0.514 A	1.028 A
TF-Luna LiDAR (peak)	1	0.150 A	0.150 A
SG90 servo (stall, conservative)	1	0.75 A	0.75 A
OLED (SSD1306, full-bright conservative)	1	0.016 A	0.016 A
OLED (SSD1362, 50% Pixels On)	1	0.035 A	0.035 A
IR sensor module (board max)	1	0.022 A	0.022 A
IMU (gyro + accel)	1	0.00072 A	0.00072 A
Total peak current			3.80 A

The total peak current is approximately

$$I_{\text{peak}} \approx 3.80 \text{ A.}$$

This value is used to verify that the battery and power distribution system can safely deliver transient high loads without excessive voltage drop or system reset.

Battery Runtime (Worst-Case)

Assuming continuous operation at the peak current, the theoretical minimum runtime for a battery of capacity C (mAh) is

$$T_{\text{worst}} = \frac{C}{1000 I_{\text{peak}}} \text{ hours.}$$

Using the selected battery capacity of $C = 2200$ mAh:

$$T_{\text{worst}} = \frac{2200}{1000 \times 3.80} \approx 0.58 \text{ h} \approx 35 \text{ min.}$$

This represents a theoretical lower bound, as stall conditions are not sustained during normal operation.

Typical (Expected) Current Budget

In realistic operation, motors run near an efficient operating point, sensors draw average current, and servo peak loads occur only briefly. Typical values therefore provide a more representative estimate of power consumption.

Table 4: Typical current budget (expected operation)

Component	Qty	Typical per unit	Subtotal
Pololu 4882 DC motor (typical, max-eff point)	2	0.17 A	0.34 A
Arduino UNO R4 WiFi (average)	2	0.094 A	0.188 A
TF-Luna LiDAR (average)	1	0.070 A	0.070 A
SG90 servo (running, no-load mid)	1	0.170 A	0.170 A
OLED (SSD1306, typical content)	1	0.010 A	0.010 A
OLED (SSD1362, typ @ checkboard spec)	1	0.025 A	0.025 A
IR sensor module (board)	1	0.022 A	0.022 A
IMU (gyro + accel)	1	0.00072 A	0.00072 A
Total typical current			0.826 A

The expected operating current is approximately

$$I_{\text{typ}} \approx 0.826 \text{ A.}$$

The corresponding runtime estimate is

$$T_{\text{typ}} = \frac{C}{1000 I_{\text{typ}}}.$$

For $C = 2200$ mAh:

$$T_{\text{typ}} = \frac{2200}{1000 \times 0.821} \approx 2.66 \text{ h.}$$

Accounting for DC–DC conversion losses, battery aging, internal resistance effects, and non-ideal operating conditions, a practical runtime of approximately 2–2.5 hours is expected.

Conclusion

The analysis indicates that the system requires a peak current of approximately 3.8 A during extreme conditions, while typical operation draws about 0.826 A. This confirms that the selected power system provides sufficient current capability and energy capacity to support reliable operation of the robot and demonstrates the technical robustness of the proposed design.

4 Project Summary

This report presented the design, modelling and implementation of a differential-drive autonomous delivery robot built on the Arduino Uno R4 platform. The system integrates infrared reflectance-based line tracking via an IR sensor array, LiDAR-based obstacle detection, and a hierarchical structured PID control architecture governing motor speed, pitch, yaw, and translational velocity.

A distributed master–slave microcontroller architecture was employed to decouple high-level sensing and navigation tasks from low-level motor control, improving computational efficiency and system reliability. Sequential PID tuning with enforced bandwidth separation ensured closed-loop stability in the presence of dynamic coupling between subsystems.

Electrical feasibility and operational robustness were validated through comprehensive component specification analysis and quantitative power budget assessment, ensuring that the selected battery and power distribution architecture meet both peak and typical operating requirements.

The resulting system demonstrates a computationally efficient and technically feasible approach to structured indoor autonomous delivery. Prioritising reliable real-time performance without reliance on computationally intensive SLAM-based localisation frameworks.

5 Electronics and Electrical Design

write i2c specifications -which communication protocols are used and why, what typical signal patterns are the components/sensors exchanging and so on

5.1 Electrical Schematics

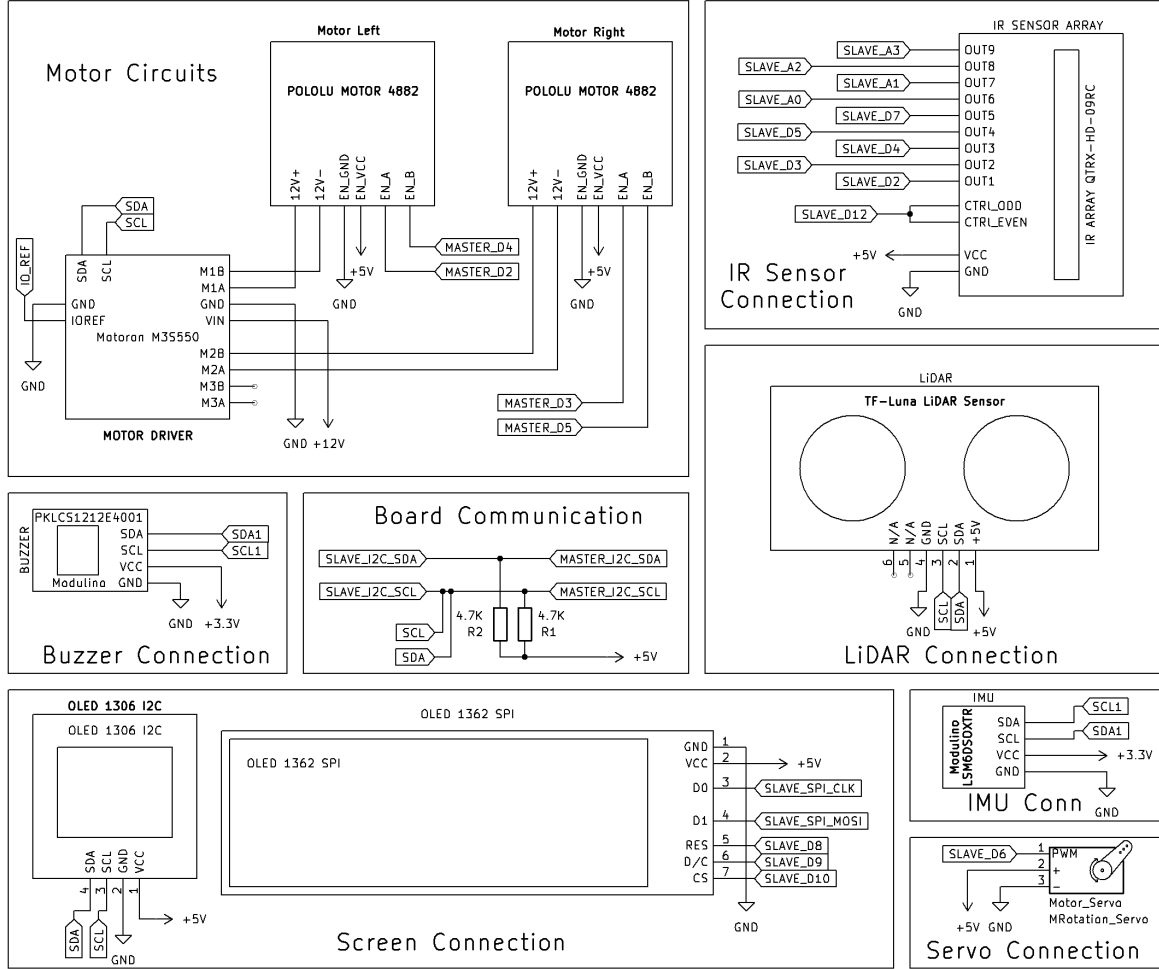


Figure 14: System Electrical Connection Diagram

Figure 14 presents the electrical schematics of the system. The architecture is divided into functional subsystems including motor control, sensing, inter-board communication, user interface, and actuation.

The motor drive subsystem consists of two Pololu 4882 DC motors controlled via a dedicated motor driver module. Direction and enable signals are provided by the master microcontroller; motor activation is supplied via a 12V DC rail.

A hierarchical control architecture is implemented, with master and slave microcontrollers communicating via an I²C interface. Pull-up resistors (4.7 k Ω) are included in the SDA and SCL lines to ensure reliable bus operation.

The sensing subsystem includes an IR sensor array, a TF-Luna LiDAR module, and a Modulino IMU. The IR sensor is interfaced to the slave controller via digital input lines, while the LiDAR and IMU communicate using serial and I²C interfaces respectively. Voltage regulation is implemented to provide both 5 V and 3.3 V supply rails as required by different peripherals.

5.2 Detailed Specifications of Key Components

TF-Luna LiDAR Specifications

Description	Parameter Value
Operating range	0.2 m – 8 m
Accuracy	± 6 cm (0.2–3 m)
	$\pm 2\%$ (3–8 m)
Measurement unit	cm
Range resolution	1 cm
Field of View (FOV)	2°
Frame rate	1–250 Hz (adjustable)
Power supply voltage	3.7V-5.2V
Average current	≤ 70 mA
Peak current	150 mA
Power consumption	≤ 350 mW
Communication signal level	LVTTL (3.3V)

Table 5: TF-Luna LiDAR Specifications

Mini Servo SG90 (180° Resin Gear) Specifications

Description	Parameter Value
Product	SG90 Micro Servo
Torque	2.0 kg·cm (4.8 V)
	2.2 kg·cm (6 V)
Speed	0.09 s/60° (4.8 V)
	0.08 s/60° (6 V)
Rotate angle	180°
Operating voltage	4.8–6 V
Gear type	Plastic (resin)
Dead band width	7 μ s
Weight	10.5 g
Dimensions	22.8 mm $\times$ 12.2 mm $\times$ 28.5 mm

Table 6: SG90 Micro Servo Specifications

Pololu IR Sensor Specifications

Description	Parameter Value
Pitch $\times$ Sensors	4 mm $\times$ 9
Module size	37.0 mm $\times$ 20.0 mm
Output type	RC (digital)
Max current (LED)	3.5 mA
Max current (board)	22 mA
Optimal sensing range	10 mm

Table 7: IR Line Sensor Module Specifications

IMU (LSM6DSOXTR) Specification

Description	Parameter Value
Sensor	LSM6DSOXTR
Supply voltage	1.71 V – 3.6 V
Power consumption	Accel: 170 μ A
	Gyro: 0.55 mA
Gyroscope range	± 125 dps – ± 2000 dps
Accuracy	Accel: ± 20 mg
	Gyro: ± 1 dps
Resolution	Accel: 0.061 mg/LSB – 0.488 mg/LSB
	Gyro: 4.375 mdps/LSB – 70 mdps/LSB
Communication	I ² C

Table 8: IMU (LSM6DSOXTR) Specifications

OLED 1306 Specifications

Characteristics	Symbol	Min	Typ	Max	Unit
Supply Voltage for Logic	V_{DD}	1.65	2.8	3.3	V
Supply Voltage for Display (Supplied Externally)	V_{CC}	6.4	–	9.0	V
Supply Voltage for DC/DC	V_{BAT}	3.5	–	4.2	V
Supply Voltage for Display (Generated by Internal DC/DC)	V_{CC}	7.0	7.25	7.5	V
High Level Input	V_{IH}	$0.8V_{DD}$	–	V_{DD}	V
Low Level Input	V_{IL}	0	–	$0.2V_{DD}$	V
High Level Output	V_{OH}	$0.9V_{DD}$	–	V_{DD}	V
Low Level Output	V_{OL}	0	–	$0.1V_{DD}$	V
Operating Current for V_{DD}	I_{DD}	–	180	300	μA
Operating Current for V_{CC} (Externally Supplied)	I_{CC}	–	10	16	mA
Operating Current for V_{BAT} (Internal DC/DC)	I_{BAT}	–	23.0	29.0	mA
Sleep Mode Current for V_{DD}	$I_{DD,SLEEP}$	–	1	5	μA
Sleep Mode Current for V_{CC}	$I_{CC,SLEEP}$	–	2	10	μA

Table 9: OLED 1306 Display Specifications

OLED 1362 Specifications

Item	Symbol	Min	Typ	Max	Unit
Supply Voltage for Display	V_{CC}	10.0	12.0	12.5	V
Low Voltage Power Supply	V_{CI}	1.65	–	3.5	V
Power Supply for I/O Pins	V_{DDIO}	1.65	–	V_{CI}	V
Logic Supply Voltage	V_{DD}	1.65	–	2.6	V
High Level Input	V_{IH}	$0.8V_{DDIO}$	–	–	V
Low Level Input	V_{IL}	–	–	$0.2V_{DDIO}$	V
High Level Output	V_{OH}	$0.9V_{DDIO}$	–	–	V
Low Level Output	V_{OL}	–	–	$0.1V_{DDIO}$	V
Display Current (50% Pixels On)	I_{CC}	–	25	35	mA

Table 10: OLED 1362 Display Specifications

Pololu Geared Motor 4882 Specifications

Description	Parameter Value
Motor type	Low Power (12V)
Gear ratio	9.68:1
No-load speed	580 RPM
No-load current	60 mA $\pm$ 50%
Speed (max eff)	480 RPM
Torque (max eff)	1.9 kg·mm
Current (max eff)	0.17 A
Output power	0.94 W
Efficiency	47%
Max power	1.8 W
Stall torque	13 kg·mm
Stall current	0.9 A

Figure 15: Gearmotors Parameters Table⁸



Figure 16: Gearmotors Performance Summary Graph⁸

Pololu Motoron M3S550 Specifications

Description	Parameter Value
Product	Triple Motor Controller Shield
Control interface	I ² C (up to 400 kHz)
Motor channels	3 (independent)
Motor supply voltage (VIN)	1.8 V – 22 V
Recommended max nominal battery voltage	16 V
Max continuous current per channel	1.7 A
Logic voltage range (VDD/IOREF)	3.1 V – 5.5 V
PWM frequency options (kHz)	1, 2, 4, 5, 10, 20, 40, 80
Operating temperature	–20°C to +85°C (typical)

Table 11: Motoron M3S550 Triple Motor Controller Specifications⁹

5.3 Mechanical Structure and Actuation Design

6 Experimental Evaluation

7 Discussion and Conclusion

8

Bibliography

- [1] BIOGARD, “Automated serving robot,” [Online]. Available: <https://www.biogardasia.com/serving-robot/>, Accessed: Feb. 24, 2026.
- [2] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA, USA: MIT Press, 2005.
- [3] MathWorks, “SLAM,” [Online]. Available: <https://uk.mathworks.com/discovery/slam.html>, Accessed: Feb. 24, 2026.
- [4] anova9347, “Line follower robot (with pid controller),” [Online]. Available: <https://projecthub.arduino.cc/anova9347/line-follower-robot-with-pid-controller-01813f>, 2020, Accessed: Feb. 24, 2026.
- [5] P. Cheung, “Servo motor sg90 data sheet,” [Online]. Available: http://www.ee.ic.ac.uk/pcheung/teaching/DE1_EE/stores/sg90_datasheet.pdf, Accessed: Feb. 24, 2026.
- [6] Benewake (Beijing) Co., Ltd., *TF-Luna LiDAR Product Manual*, [Online]. Available: https://files.seeedstudio.com/wiki/Grove-TF_Mini_LiDAR/res/SJ-PM-TF-Luna-A03-Product-Manual.pdf, 2021, Accessed: Feb. 24, 2026.
- [7] Wikipedia contributors, “I2C,” [Online]. Available: <https://en.wikipedia.org/wiki/I2C>, Accessed: Feb. 24, 2026.
- [8] Pololu Corporation Inc., “Micro metal gearmotors performance summary (rev. 6.1),” [Online]. Available: <https://www.pololu.com/file/0J1487/pololu-micro-metal-garmotors-rev-6-1.pdf>, Tech. Rep.
- [9] Pololu, *Pololu Motoron Motor Controller User’s Guide*, [Online]. Available: <https://www.pololu.com/docs/0J84/all>, Accessed: Feb. 24, 2026.